

Тестирование

Доказываем поведение программы и защищаем его от повторных ошибок.

Главная идея

Тест автоматически выполняет код с заданными входами и сравнивает результат с ожиданием. Хороший тест проверяет одно понятное поведение.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
def test_calculate_total():  
    assert calculate_total(150, 3) == 450
```

План урока

Один блок теории - сразу практика и проверка.

1

Понять смысл

2

Выучить
термины

3

Разобрать код

4

Попробовать
самому

5

Собрать мини-
задачу

Как заниматься

Не читай как книгу. На практических слайдах остановись и попробуй решить сам.

Смысл темы простыми словами

Сначала бытовая идея, потом код.

Идея 1

Тест автоматически выполняет код с заданными входами и сравнивает результат с ожиданием. Хороший тест проверяет одно понятное поведение.

Идея 2

Unit-тест изолирует небольшую функцию, integration-тест проверяет взаимодействие нескольких частей. Пирамида тестов обычно содержит много быстрых unit-тестов и меньше крупных сценариев.

Идея 3

assert формулирует ожидание. Mock временно заменяет внешнюю зависимость, но чрезмерное количество mock может дать ложную уверенность. Coverage показывает выполненные тестами строки, а...

Термины дня

Эти слова нужно узнавать в коде и документации.

test case

один автоматический сценарий проверки поведения

unit test

изолированная проверка небольшой единицы кода

integration test

проверка совместной работы нескольких компонентов

assert

утверждение об ожидаемом результате

mock

контролируемая замена внешней зависимости в тесте

coverage

метрика кода, выполненного во время тестов

Код дня

Минимальный пример без лишней сложности.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
def test_calculate_total():  
    assert calculate_total(150, 3) == 450
```

Что ищем

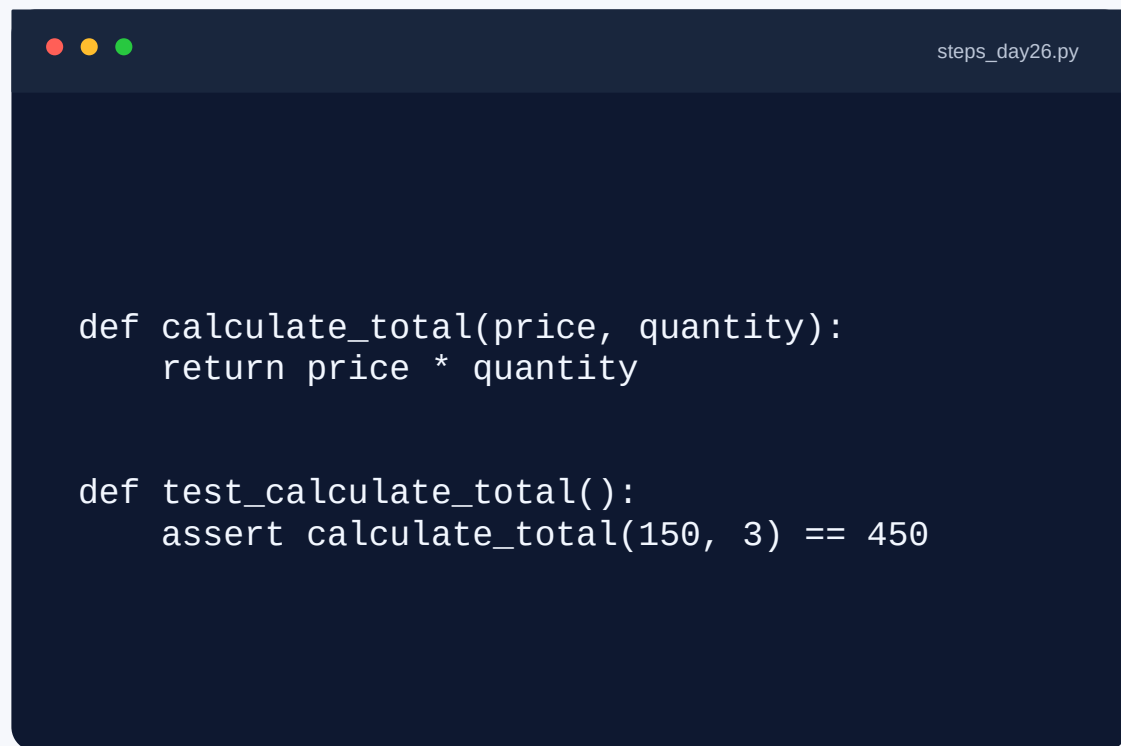
Где данные? Где действие? Где результат?

Главный вопрос

Можешь объяснить каждую строку обычным языком?

Разбор кода по шагам

Не переписываем вслепую.



```
def calculate_total(price, quantity):  
    return price * quantity  
  
def test_calculate_total():  
    assert calculate_total(150, 3) == 450
```

- Прочитай строку сверху вниз.
- Назови данные, с которыми работает код.
- Объясни, какое действие выполняется.
- Проверь, какой результат должен получиться.



Важно

Если строка непонятна - остановись и переведи ее на обычный язык.

Практика: что произойдет?

Ответь без запуска кода.

```
question_day26.py

def calculate_total(price, quantity):
    return price * quantity

def test_calculate_total():
    assert calculate_total(150, 3) == 450
```

Вопрос

Что программа сделает после запуска?

Подсказка

Ищи последовательность: входные данные -> действие -> результат.

Ответ и объяснение

Сравни со своим рассуждением.

Ожидаемая логика

Этот пример относится к теме: Тестирование. Доказываем поведение программы и защищаем его от повторных ошибок.

Почему так?

Python выполняет инструкции по порядку. Поэтому важно читать код сверху вниз.

Проверка себя

Если можешь объяснить без подсказки - тема закрепляется.

Практика: сделай руками

Небольшое задание по теме дня.

Задание

Напиши 3 теста для `calculate_total`.

Шаг 1

Создай простой файл или черновик.

Шаг 2

Напиши минимальный рабочий вариант.

Шаг 3

Запусти и проверь результат.

Мини-тест

Выбери ответ и объясни почему.

Вопрос

Гарантирует ли 100% coverage отсутствие ошибок?

A

Да, всегда

B

Нет, coverage не оценивает качество ожиданий

C

Да, если проект на Python

D

Только без unit-тестов

Ответ мини-теста

Проверяем не память, а понимание.

Правильный ответ

В: Нет, coverage не оценивает качество ожиданий

Объяснение

Покрытие показывает выполнение строк, но тест может не проверять важный результат или пограничный случай.

Частая ошибка новичка

Лучше заранее увидеть, где обычно ломается.

Ошибка

Пытаться запомнить формулировку, но не понимать, где это видно в коде.

Как исправить

После каждого термина придумай свой пример и найди его в коде.

Правило

Термин считается выученным, когда ты можешь объяснить его на бытовом примере.

Самостоятельная мини-задача

Напиши маленький код или схему почти с нуля.

Условие

Напиши 3 теста для `calculate_total`.

- Что программа получает?
- Что она должна сделать?
- Какой результат показать?
- Какие термины дня здесь применяются?

Разбор решения

После попытки сравни с этим подходом.

```
def calculate_total(price, quantity):  
    return price * quantity  
  
def test_calculate_total():  
    assert calculate_total(150, 3) == 450
```

Не один вариант

Твой код может отличаться. Главное - он решает задачу и читается.

Проверка

Запусти пример и объясни результат вслух.

Задание со звездочкой

Немного сложнее, но без новой темы.

Условие

Расширь практику дня: добавь проверку ошибки, 2-3 тестовых примера и короткое объяснение для README.

Подсказка

Не усложняй. Сделай маленькое улучшение и обязательно проверь его на примере.

Итог дня

Что должно остаться после урока.

- объяснять тему "Тестирование" простыми словами
- узнавать главные термины дня
- читать короткий пример кода
- выполнять практику руками
- находить одну типовую ошибку
- формулировать следующий маленький шаг

Домашнее задание

Напишите три теста для `calculate_total`: обычные значения, нулевое количество и отрицательное значение, если оно запрещено правилами.